

Classification, Decision Trees & Regression Summary with Answers

1. Classification Basics

****Goal:**** Learn a model that maps input features (x) → output categories (y).

****Training vs Test:**** Training fits the model; Test evaluates it.

****Independent vs Dependent:**** X are predictors; y is target.

****Examples:**** Tumor type (benign/malignant), loan default (Yes/No).

****Difference from Regression:**** Classification = discrete; Regression = continuous.

2. Decision Tree Structure

Decision node → test attribute; Leaf node → final class.

Non-parametric: no predefined equation.

Overfitting → tree too complex; Pruning reduces it.

Different trees can classify same data correctly.

3. Impurity Measures

Entropy, Gini, and Error measure homogeneity.

Measure	Formula	Result Example
---------	---------	----------------

Entropy	$-\sum p \cdot \log(p)$	1.571
---------	-------------------------	-------

Gini	$1 - \sum p^2$	0.66
------	----------------	------

Error	$1 - \max(p)$	0.60
-------	---------------	------

Lower value = more pure node.

4. Splitting and Attribute Types

- Binary split: 2 groups (Income > 80K?).

- Multi-way: all unique values.

Nominal → unordered, Ordinal → keep order, Continuous → threshold split.

Discretization = converting continuous into ordered ranges.

5. Tree Induction Algorithms

Algorithm	Impurity	Note
-----------	----------	------

ID3	Entropy	early model
-----	---------	-------------

C4.5	Entropy + Gain Ratio	improved ID3
------	----------------------	--------------

CART	Gini	sklearn default
------	------	-----------------

Greedy = best local split only, no backtracking.

Advantages: easy to interpret, handle mixed data, non-linear relations.

6. Ensemble Methods

Bagging vs Boosting

Type	Idea	Example
------	------	---------

Bagging	random subsets averaged	Random Forest
---------	-------------------------	---------------

Boosting	sequential, fix previous errors	AdaBoost, XGBoost
----------	---------------------------------	-------------------

Random Forest → less overfitting.

7. Linear Regression + GridSearchCV

Regression = continuous prediction.

Scaling normalizes features.

GridSearchCV = find best parameters with cross-validation.

Cross-validation prevents overfitting.

8. Pandas Essentials

```
```python
import pandas as pd
df = pd.read_csv('data.csv')
df = df.dropna()
df.select_dtypes(include='number')
df[df['Income'] > 80000]
df['Income'].fillna(df['Income'].mean(), inplace=True)
```
```

.loc[] → label-based, .iloc[] → index-based.

9. Real-world Mini Challenge

| Age | Salary | OwnsCar | BuysInsurance |

|-----|-----|-----|-----|

| 25 | 30K | No | No |

| 40 | 80K | Yes | Yes |

| 35 | 60K | No | Yes |

| 28 | 40K | No | No |

| 50 | 90K | Yes | Yes |

Entropy(root) = 0.971 (Yes=3, No=2).

Best split = OwnsCar.

New case (38y, 70K, NoCar) → BuysInsurance = Yes.